

Fundamentos de Analítica de Datos Financieros

Parte II | Econometría Financiera

Víctor Macías E.

En estas notas se usarán las siguientes librerías de Python:

```
1 import numpy as np
2 import pandas as pd
3 import scipy.stats as stats
4 import matplotlib.pyplot as plt
```

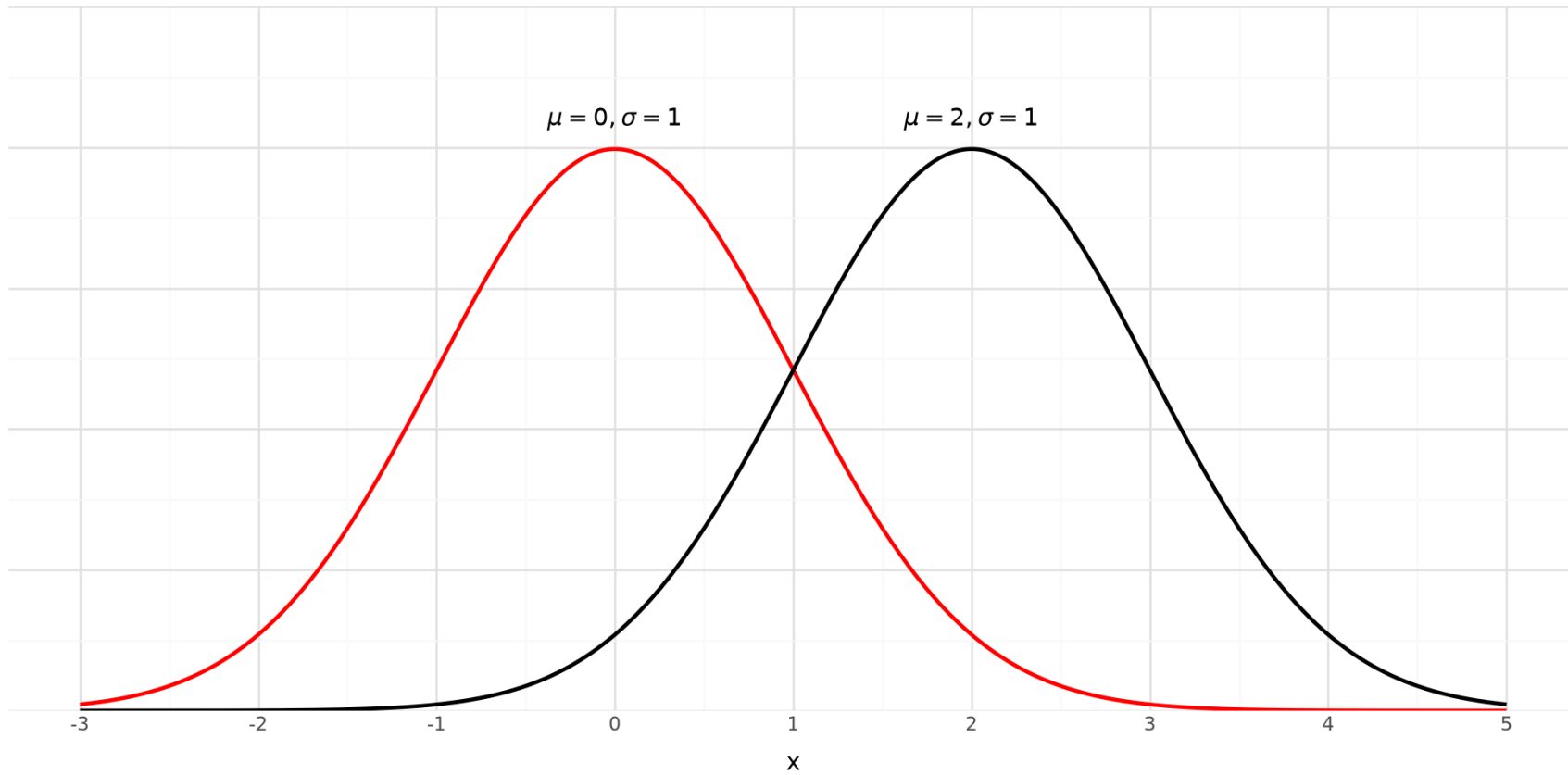
Distribución normal

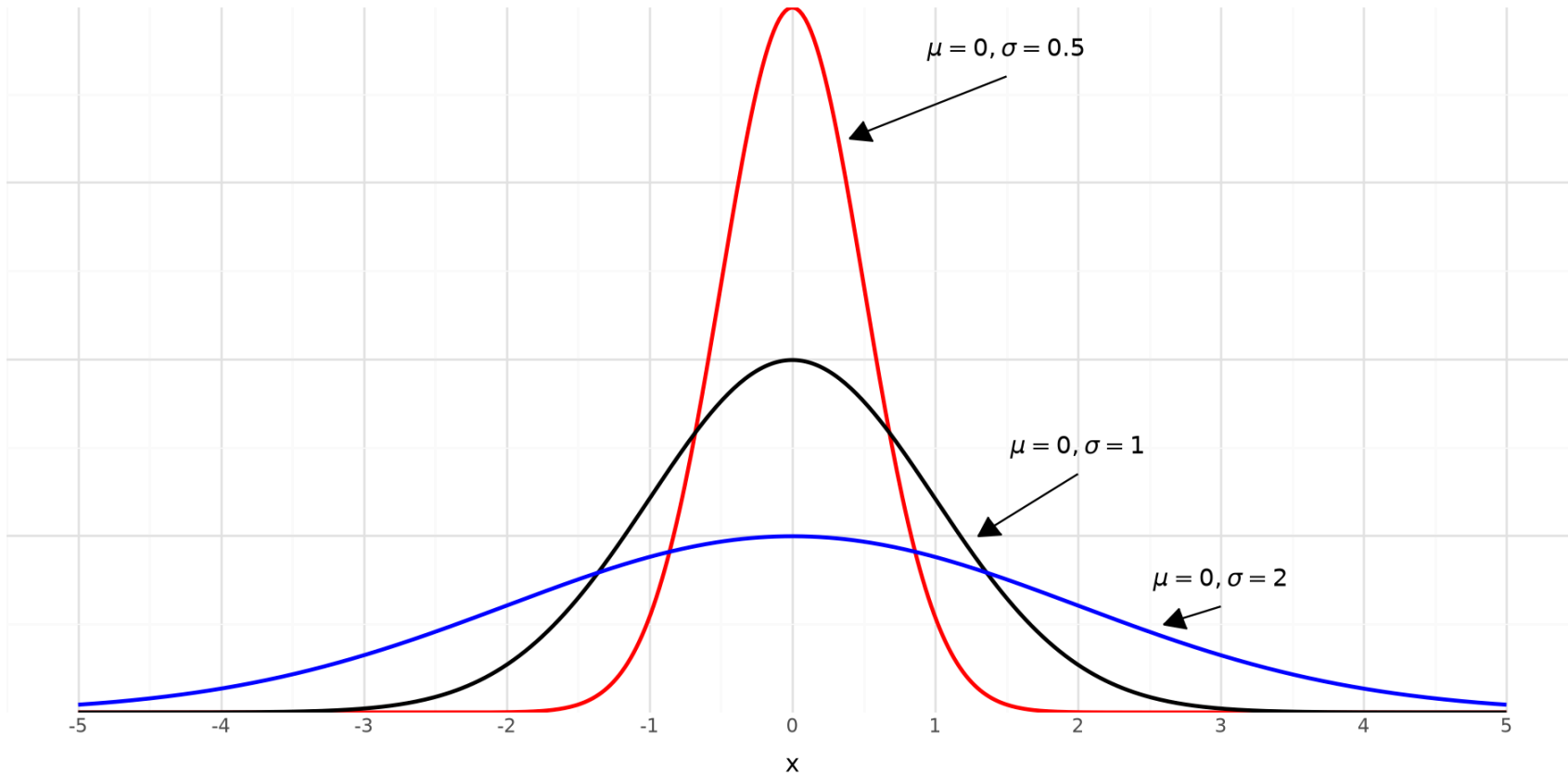
- La distribución de muchas variables aleatorias puede representarse por la distribución normal
- De acuerdo al Teorema del Límite Central la distribución muestral de una media o una proporción es normal.
- Se usará la notación $N(\mu, \sigma^2)$ para especificar una distribución normal con media μ y varianza σ^2 .
- La distribución normal es simétrica y unimodal.

- La función de densidad de probabilidad de una variable X con distribución normal es:

$$f(x; \mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \quad -\infty < x < \infty$$

- La distribución normal estándar es la distribución normal con $\mu = 0$ y $\sigma^2 = 1$ y se expresa como $N(0, 1)$.





- Se entiende por estandarización de una variable al método que permite transformar los valores de X a Z , donde:

$$Z = \frac{X - \mu}{\sigma}$$

- El valor de Z se denomina Z-score. Por ejemplo, una observación con un Z-score de 2 corresponde a una que se encuentra dos desviaciones estándar arriba de la media. El Z-score es independiente de la unidad de medición.

- Si $X \sim N(\mu, \sigma^2)$, entonces Z tiene una distribución normal estándar que tiene $\mu = 0$ y $\sigma^2 = 1$.

$$\begin{aligned} P(a \leq X \leq b) &= P\left(\frac{a - \mu}{\sigma} \leq \frac{X - \mu}{\sigma} \leq \frac{b - \mu}{\sigma}\right) \\ &= \Phi\left(\frac{b - \mu}{\sigma}\right) - \Phi\left(\frac{a - \mu}{\sigma}\right) \end{aligned}$$

donde $\Phi()$ representa la distribución normal estándar acumulada.

Distribución normal

Densidad: `stats.norm.pdf(x, loc, scale)`

Probabilidad acumulada: `stats.norm.cdf(q, loc, scale)`

Percentil: `stats.norm.ppf(p, loc, scale)`

Números pseudoaleatorios: `np.random.normal(loc, scale, size)`

Ejemplo 1:

Si $x \sim N(0, 1)$, calcula con tres decimales:

$$P(x \leq 0) = ?$$

```
1 round(stats.norm.cdf(x = 0, loc = 0, scale = 1), 3)
```

```
np.float64(0.5)
```

$$P(x \leq 1.96) = ?$$

```
1 round(stats.norm.cdf(x = 1.96, loc = 0, scale = 1), 3)
```

```
np.float64(0.975)
```

$$P(x \leq -1.96) = ?$$

```
1 round(stats.norm.cdf(x = -1.96, loc = 0, scale = 1), 3)
```

```
np.float64(0.025)
```

$$P(-3 \leq x \leq 3) = ?$$

```
1 round(stats.norm.cdf(x = 3, loc = 0, scale = 1), 3) - round(stats.norm.cdf(x = -3, loc = 0, scale = 1), 3)
```

```
np.float64(0.998)
```

Ejemplo 2:

- El percentil 95 de la distribución normal estándar es:

```
1 round(stats.norm.ppf(0.95, loc = 0, scale = 1), 3)
```

```
np.float64(1.645)
```

- El percentil 97.5 de la distribución normal estándar es:

```
1 round(stats.norm.ppf(0.975, loc = 0, scale = 1), 3)
```

```
np.float64(1.96)
```

- El percentil 99 de la distribución normal estándar es:

```
1 round(stats.norm.ppf(0.99, loc = 0, scale = 1), 3)
```

```
np.float64(2.326)
```

Ejemplo 3:

Genera 10000 números pseudoaleatorios de una distribución normal estándar. Calcula su sesgo (skewness) y curtosis.

```
1 # Generar los 10000 números
2 np.random.seed(1479)
3 muestra = np.random.normal(0, 1, 10000)
4
5 # Calcular métricas usando stats directamente
6 sesgo = stats.skew(muestra)
7 curtosis = stats.kurtosis(muestra) # Curtosis en exceso (referencia = 0)
8 curtosis_normal = stats.kurtosis(muestra, fisher=False)
9
10 print(f"Sesgo: {sesgo:.1f}")
11 print(f"Curtosis (exceso): {curtosis:.1f}")
12 print(f"Curtosis normal: {curtosis_normal:.1f}")
```

Sesgo: 0.0

Curtosis (exceso): 0.0

Curtosis normal: 3.0



Observa que el sesgo de la distribución normal es 0 y la curtosis es 3.

Ejemplo 4:

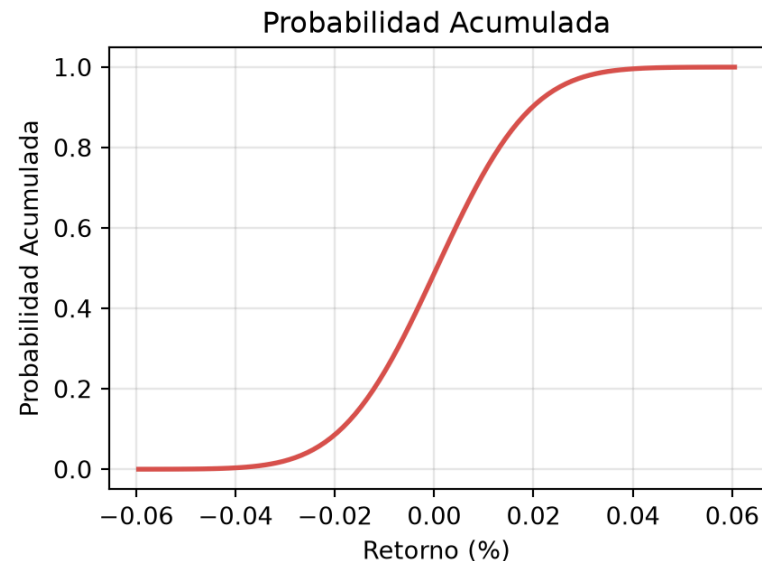
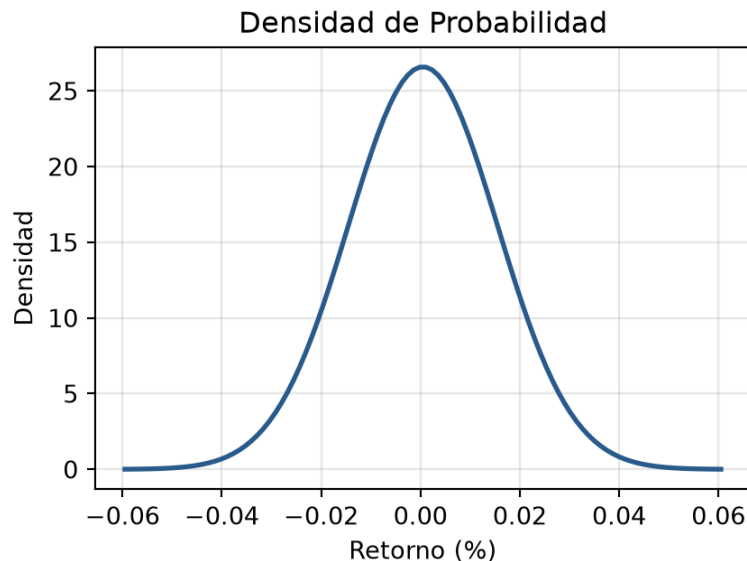
Los retornos de un activo siguen una distribución normal. El retorno diario esperado es 0.05% y la volatilidad diaria (desviación estándar) es 1.5%

- a. Grafica la densidad de probabilidad
- b. Grafica la función de distribución acumulada
- c. Calcula la probabilidad de perder más de un 2% en un día
- d. Calcula la probabilidad de tener un retorno positivo en un día

```

1 # Retorno diario esperado = 0.05%, Volatilidad diaria = 1.5%
2 mu, sigma = 0.0005, 0.015
3 x = np.linspace(mu - 4*sigma, mu + 4*sigma, 100)
4
5 pdf = stats.norm.pdf(x, mu, sigma) # densidad
6 cdf = stats.norm.cdf(x, mu, sigma) # distribución acumulada
7
8 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(9, 3.5)) # 2 subplots (1 fila, 2 columnas)
9
10 # Gráfico 1: Función de densidad de probabilidad (pdf)
11 ax1.plot(x, pdf, color='#2b5c8f', lw=2)
12 ax1.set_title("Densidad de Probabilidad")
13 ax1.set_xlabel("Retorno (%)")
14 ax1.set_ylabel("Densidad")
15 ax1.grid(True, alpha=0.3)
16
17 # Gráfico 2: Función de distribución acumulada (cdf)
18 ax2.plot(x, cdf, color='#d9534f', lw=2)
19 ax2.set_title("Probabilidad Acumulada")
20 ax2.set_xlabel("Retorno (%)")
21 ax2.set_ylabel("Probabilidad Acumulada")
22 ax2.grid(True, alpha=0.3)
23
24 plt.tight_layout()
25 plt.show()

```



c. La probabilidad de perder más de un 2% en un día es:

```
1 # Cálculo de probabilidades en Python para un retorno X < -2%
2 prob_pérdida = stats.norm.cdf(-0.02, loc=0.0005, scale=0.015)
3 print(f"Probabilidad de perder más de 2% en un día: {prob_pérdida:.2%}")
```

Probabilidad de perder más de 2% en un día: 8.59%

d. La probabilidad de un retorno positivo en un día es:

```
1 # Cálculo de probabilidades en Python para un retorno X < -2%
2 prob_retorno_positivo = 1 - stats.norm.cdf(0, loc=0.0005, scale=0.015)
3 print(f"Probabilidad de un retorno positivo en un día: {prob_retorno_positivo:.2%}")
```

Probabilidad de un retorno positivo en un día: 51.33%

t-Student

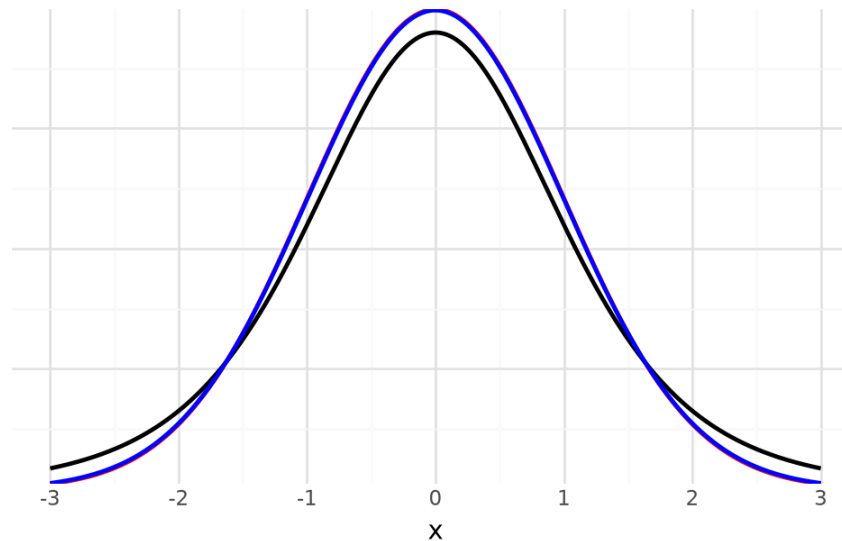
Densidad: `stats.t.pdf(x, df, loc, scale)`

Probabilidad acumulada: `stats.t.cdf(q, df, loc, scale)`

Percentil: `stats.t.ppf(p, df, loc, scale)`

Números pseudoaleatorios: `stats.t.rvs(df, loc, scale, size)`

- Distribución normal estándar (línea roja)
- Distribución t-Student con 5 grados de libertad (línea negra) y 120 grados de libertad (línea azul).



Nota que la línea azul se superpone a la roja.

Riesgo financiero: Normal vs. t -Student

Los mercados reales presentan colas pesadas (fat tails) y exceso de curtosis. Esto significa que las crisis y las pérdidas extremas ocurren con mucha mayor frecuencia de la que predice una distribución Normal.

La distribución t -Student permite capturar mejor este riesgo al dar mayor probabilidad a los eventos extremos.

Value at Risk (VaR)

¿Qué nivel de pérdida es tal que estamos un $X\%$ confiados en que no será superado durante un periodo determinado?

Ejemplo 5:

Suponga que la ganancia de un portafolio durante seis meses sigue una distribución normal con una media de \$2 millones y una desviación estándar de \$10 millones. Calcula el VaR para el portafolio con un horizonte de 6 meses y un nivel de confianza de 99%.

```
1 print(f"VaR: {-2 - stats.norm.ppf(0.01, loc= 0, scale= 1)*10:.1f} millones")
```

VaR: 21.3 millones

Otra forma de resolverlo es:

```
1 print(f"VaR: {-stats.norm.ppf(0.01, loc= 2, scale= 10):.1f} millones")
```

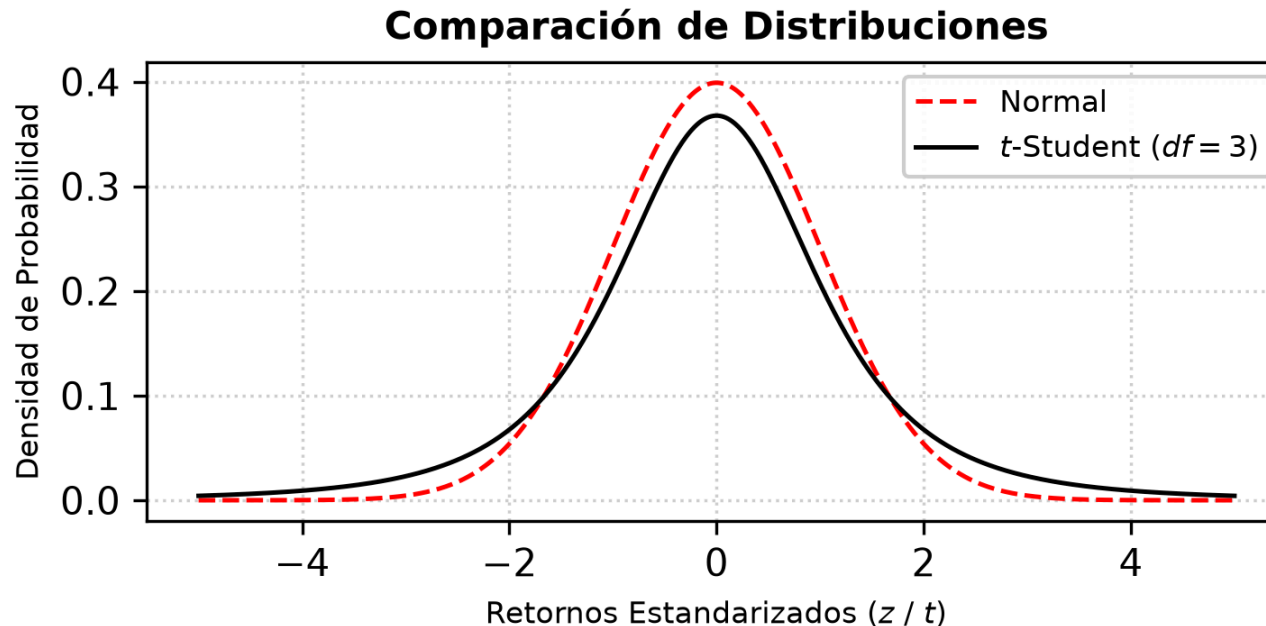
VaR: 21.3 millones

Por lo tanto, solo hay un 1% de probabilidad de sufrir una pérdida superior 21.3 millones en un período de 6 meses.

```

1 # 1. Generar eje X (retornos estandarizados)
2 x = np.linspace(-5, 5, 500)
3
4 # 2. Calcular densidades
5 norm_pdf = stats.norm.pdf(x)
6 t_pdf = stats.t.pdf(x, df=3)
7
8 # 3. Graficar
9 plt.figure(figsize=(5, 2.6), dpi=150)
10 plt.plot(x, norm_pdf, "r--", linewidth=1.2, label="Normal")
11 plt.plot(x, t_pdf, "k-", linewidth=1.2, label="$t$-Student ($df=3$)")
12 plt.title("Comparación de Distribuciones", fontsize=10, fontweight='bold')
13 plt.xlabel("Retornos Estandarizados ($z$ / $t$)", fontsize=8)
14 plt.ylabel("Densidad de Probabilidad", fontsize=8)
15 plt.legend(frameon=True, facecolor='white', framealpha=0.9, fontsize=8)
16 plt.grid(True, linestyle=':', alpha=0.6)
17 plt.tight_layout()
18 plt.show()

```



- Percentil 1:

```
1 print(f"Valor de z: {stats.norm.ppf(0.01, loc = 0, scale = 1):.3f}") # Distribución normal
```

Valor de z: -2.326

```
1 print(f"Valor de t (3 grados de libertad): {stats.t.ppf(0.01, df=3):.3f}") # Distribución t-Student
```

Valor de t (3 grados de libertad): -4.541

- Percentil 5:

```
1 print(f"Valor de z: {stats.norm.ppf(0.05, loc = 0, scale = 1):.3f}") # Distribución normal
```

Valor de z: -1.645

```
1 print(f"Valor de t (3 grados de libertad): {stats.t.ppf(0.05, df=3):.3f}") # Distribución t-Student
```

Valor de t (3 grados de libertad): -2.353